2.1 Concordancing

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 30 September 2026, 19:54

This unit introduces concordancing with R using the `quanteda` package, demonstrating keyword-in-context searches, dispersion analysis, and data export for transparent corpus-linguistic research.

Table of contents

Recommended reading . . . 1
Preparation . . . 2
Concordancing . . . 3
- Simple queries . . . 3
- Multi-word queries . . . 4
- Multiple simultaneous queries . . . 4
- Window size . . . 5
- Saving your output . . . 6

Characterising the output . . . 6
Advanced techniques . . . 8
- Word frequencies . . . 8
- Removing markup . . . 9
- Zipf's Law . . . 11
- Lexical diversity . . . 13

Alternative concordancing software . . . 17

References **17**

Recommended reading

- In-depth introduction to concordancing with R:

  Schweinberger (2024)

- Natural Language Processing (NLP) with `quanteda`:

  Benoit et al. (2018)

  Online reference

- On corpus-linguistic theory:

  Wulff and Baker (2020)

  Lange and Leuckert (2020)

  McEnery, Xiao, and Yukio (2006)

Preparation

> Script
>
> You can find the full R script associated with this unit here.

> Working directory
>
> To ensure that R can correctly access your data, it is crucial to set the working directory appropriately (see also 1.6 Importing/Exporting). The simplest approach is to create an **R project** in a directory of your choice and open it (see the small blue symbol in the top-right corner of the RStudio interface). If this is not feasible, or if you prefer to work outside an R project, you can follow these steps:
>
> 1. Make sure your R-script **and** the corpus (e.g., 'ICE-GB') are stored in the **same folder** on your computer.
> 2. In RStudio, go to the `Files` pane (usually in the bottom-right corner) and navigate to the location of your script. Alternatively, you can click on the three dots `...` and use the file browser instead.
> 3. Once you're in the correct folder, click on the blue icon.
> 4. Select `Set As Working Directory`. This action will update your working directory to the folder where the file is located.

In addition, make sure you have installed `quanteda` and any other packages you're planning to use. Load them at the beginning of your script:

```
library(quanteda) # Package for Natural Language Processing in R
library(quanteda.textplots) # supplementary package
library(quanteda.textstats) # supplementary package
library(writexl) # exporting data in MS Excel format
library(tidyverse) # data manipulation (needed for the advanced section)
```

To load a corpus object into R, place it in your working directory and read it into your working environment with `readRDS()`.[1]

```
# Load corpus from directory
ICE_GB <- readRDS("ICE_GB.RDS")
```

If you encounter any error messages at this stage, ensure you followed steps 1 and 2 in the callout box above.

Concordancing

A core task in corpus-linguistic research involves finding occurrences of a single word or multi-word sequence in the corpus. Lange & Leuckert (2020: 55) explain that specialised software typically "provide[s] the surrounding context as well as the name of the file in which the word could be identified." Inspecting the context is particularly important in comparative research, as it may be indicative of distinct usage patterns.

Simple queries

To obtain such a **keyword in context (KWIC)** in R, we use the `kwic()` function. We supply the corpus as well as the keyword we're interested in:

```
query1 <- kwic(ICE_GB, pattern = "eat")
```

The output in `query1` contains **concordance lines** that list all occurrences of the keyword, including the document, context to the left, the keyword itself, and the context to the right. The final column reiterates our search expression.

```
head(query1)
```

```
Keyword-in-context with 6 matches.
  [ICE_GB/S1A-006.txt, 785]           So I' d rather | eat |
 [ICE_GB/S1A-009.txt, 1198]              I must <, > | eat |
  [ICE_GB/S1A-010.txt, 958]         to <, > actually | eat |
  [ICE_GB/S1A-018.txt, 455] order one first and then | eat |
  [ICE_GB/S1A-018.txt, 498]  A > The bargain hunting | eat |
 [ICE_GB/S1A-023.txt, 1853]       B > Oh name please | eat |

 beforehand just to avoid uh
```

[1] The `ICE_GB.RDS` file you've been provided with has been pre-processed and saved in this specific format for practical reasons. It contains the corpus as a `quanteda` **tokens** object, i.e., every text has already been split up into individual tokens (words, punctuation marks, etc.).

docname	from	to	pre	keyword	post
ICE_GB/S1A-006.txt	785	785	So I ' d rather	eat	beforehand just to avoid uh
ICE_GB/S1A-009.txt	1198	1198	I must < , >	eat	them < ICE-GB:S1A-009 #
ICE_GB/S1A-010.txt	958	958	to < , > actually	eat	it for one ' s
ICE_GB/S1A-018.txt	455	455	order one first and then	eat	it and then sort of
ICE_GB/S1A-018.txt	498	498	A > The bargain hunting	eat	< ICE-GB:S1A-018 #29 : 1
ICE_GB/S1A-023.txt	1853	1853	B > Oh name please	eat	something < , , >

```
them < ICE-GB:S1A-009#71:
it for one' s
it and then sort of
< ICE-GB:S1A-018#29: 1
something <,, >
```

For a full screen display of the KWIC data frame, try `View()`:

```
View(query1)
```

> What are all these symbols in the context columns?
>
> The ICE corpora contain **markup**, i.e., additional information enclosed in angle brackets `< >`. For example, `< ICE-GB:S1A-009 #71 : 1 : A >` marks the beginning of text unit no. 71 in file S1A-009, which is uttered by speaker A, and `< , >` indicates a short pause. When reading concordance lines, this information can be very useful, so we will leave it in for now. When counting words, however, it becomes a nuisance (see Advanced techniques below).

Multi-word queries

If the search expression exceeds a single word, we need to mark it as a multi-word sequence by means of the `phrase()` function. For instance, if we were interested in the pattern *eat a*, we'd have to adjust the code as follows:

```
query2 <- kwic(ICE_GB, pattern = phrase("eat a"))
```

```
View(query2)
```

Multiple simultaneous queries

A very powerful advantage of `quanteda` over traditional corpus software is that we can query a corpus for a multitude of keywords **at the same time**. Say, we need our output

docname	from	to	pre	keyword	post
ICE_GB/S1A-059.txt	2230	2231	1 : B > I	eat a	< , > very balanced
ICE_GB/W2B-014.txt	1045	1046	: 1 > We can't	eat a	lot of Welsh or Scott
ICE_GB/W2B-022.txt	589	590	have few labour-saving devices ,	eat a	diet low in protein ,

docname	from	to	pre	keyword	post
ICE_GB/S1A-006.txt	785	785	So I ' d rather	eat	beforehand just to avoid uh
ICE_GB/S1A-009.txt	869	869	: A > Do you	drink	quite a lot of it
ICE_GB/S1A-009.txt	1198	1198	I must < , >	eat	them < ICE-GB:S1A-009 #71
ICE_GB/S1A-010.txt	958	958	to < , > actually	eat	it for one ' s
ICE_GB/S1A-014.txt	3262	3262	you were advised not to	drink	water in Leningrad because th
ICE_GB/S1A-016.txt	3290	3290	> I couldn't I couldn't	sleep	if I didn't read <

to contain hits for *eat*, *drink* as well as *sleep*. Instead of a single keyword, we supply a character vector `words` containing the strings of interest.[2]

```
words <- c("eat", "drink", "sleep")

query3 <- kwic(ICE_GB, pattern = words)
```

```
View(query3)
```

Window size

Some studies require more detailed examination of the preceding or following context of the keyword. We can easily adjust the `window` size to suit our needs:

```
query4 <- kwic(ICE_GB, pattern = "eat", window = 20)
```

[2]Naturally, you can choose any other variable name besides `words`.

docname	from	to	pre
ICE_GB/S1A-006.txt	785	785	#49 : 1 : A > Yeah < ICE-GB:S1A-006 #50 : 1 : A > So I ' d rat
ICE_GB/S1A-009.txt	1198	1198	< , > in in the summer < ICE-GB:S1A-009 #70 : 1 : A > I must
ICE_GB/S1A-010.txt	958	958	1 : B > You know I mean it would seem to be squandering it to <
ICE_GB/S1A-018.txt	455	455	s so < ICE-GB:S1A-018 #27 : 1 : A > What you should do is ord
ICE_GB/S1A-018.txt	498	498	second anyway so < laugh > < , > < ICE-GB:S1A-018 #28 : 1 :
ICE_GB/S1A-023.txt	1853	1853	> I can't bear it < , , > < ICE-GB:S1A-023 #121 : 1 : B > Oh na

docname	from	to	pre	keyword	post
ICE_GB/S1A-006.txt	785	785	So I ' d rather	eat	beforehand just to avoid uh
ICE_GB/S1A-009.txt	1198	1198	I must < , >	eat	them < ICE-GB:S1A-009 #
ICE_GB/S1A-010.txt	958	958	to < , > actually	eat	it for one ' s
ICE_GB/S1A-018.txt	455	455	order one first and then	eat	it and then sort of
ICE_GB/S1A-018.txt	498	498	A > The bargain hunting	eat	< ICE-GB:S1A-018 #29 : 1
ICE_GB/S1A-023.txt	1853	1853	B > Oh name please	eat	something < , , >

Saving your output

You can store your results in a spreadsheet file just as described in the unit on importing and exporting data.

- **Microsoft Excel** (.xlsx)

```
write_xlsx(query1, "myresults1.xlsx")
```

- **CSV** (.csv)

```
write.csv(query1, "myresults1.csv", row.names = FALSE)
```

As soon as you have annotated your data, you can load .xlsx files back into R with `read_xlsx()` from the `readxl` package and .csv files using the Base R function `read.csv()`.

Characterising the output

Recall our initial query of the verb *eat*, whose output we stored in `query1`:

First, we may be interested in obtaining some general information on our results:

- How many **tokens** (= individual hits) does the query return?

The `nrow()` function counts the number of rows in a data frame — these always correspond to the number of **observations** in our sample (here: 53).

```
nrow(query1)
```

```
[1] 53
```

- How many **types** (= distinct hits) does the query return?

By default, `kwic()` ignores capitalisation. This is why `query1` contains 52 instances of *eat* in lower case and 1 in upper case, which together add up to our 53 observations.

```
table(query1$keyword)
```

```
eat Eat
 52   1
```

Since *eat* and *Eat* are merely spelling variants of the same word form, it usually makes sense to convert all keywords to lower case before counting them:

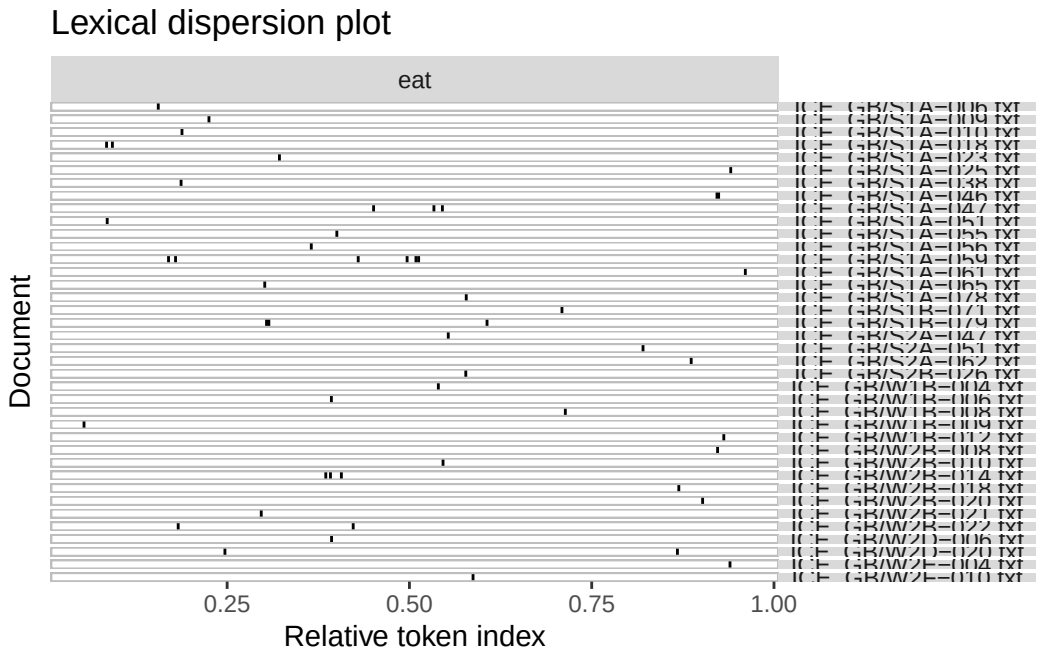
```
table(tolower(query1$keyword))
```

```
eat
 53
```

- How is the keyword distributed across corpus files?

This question relates to the notion of **dispersion**: Is a keyword spread relatively evenly across corpus files or does it only occur in specific ones? The `quanteda.textplots` package provides a very handy plotting function:

```
textplot_xray(query1)
```



It seems that *eat* occurs at least once in most text categories (both spoken and written), but seems to be much more common in face-to-face conversations (S1A). This is not surprising: It is certainly more common to discuss food in a casual chat with friends than in an academic essay (unless, of course, its main subject matter is food). Dispersion measures can thus be viewed as indicators of **contextual preferences** associated with lexemes or more grammatical patterns.

Advanced techniques

Word frequencies

We can generate a **document-feature matrix** ('dfm') by counting every single token in every ICE text.

```
dfmat_raw <- dfm(ICE_GB)

print(dfmat_raw, max_ndoc = 10, max_nfeat = 20)
```

```
Document-feature matrix of: 500 documents, 41,413 features (97.97% sparse) and 0 docvars.
                    features
docs                   < ice-gb:s1a-001 #1   :   1   a   > ok adam uhm   , what
  ICE_GB/S1A-001.txt 338            127  1 254 127  85 338  5    2  70 234   23
  ICE_GB/S1A-002.txt 347              0  1 330 116  72 347  0    3  96 197   13
  ICE_GB/S1A-003.txt 430              0  1 336 168  96 430  1    3  78 271   14
  ICE_GB/S1A-004.txt 430              0  1 292 146 123 430  5    2  70 344   20
  ICE_GB/S1A-005.txt 365              0  1 516 258 165 365  2    0  31 120   13
  ICE_GB/S1A-006.txt 397              0  1 658 329 206 397  0    0  12  65   14
  ICE_GB/S1A-007.txt 387              0  1 624 312 136 387  0    0   8  82   16
  ICE_GB/S1A-008.txt 344              0  1 590 295 210 344  2    0  19  38   15
  ICE_GB/S1A-009.txt 503              0  1 684 342 225 503  0    0  20 171    9
  ICE_GB/S1A-010.txt 474              0  1 584 292 168 474  2    0  33 230   17
                    features
docs                 did you see as missing from other activities
  ICE_GB/S1A-001.txt   4  21   6 15       2   16    13          4
  ICE_GB/S1A-002.txt   2  75   4  5       0    4     8          0
  ICE_GB/S1A-003.txt   3  69   4 13       0    3     4          2
  ICE_GB/S1A-004.txt  11  25   1 12       0    4    10          0
  ICE_GB/S1A-005.txt   5  43   6  7       0    7     2          0
  ICE_GB/S1A-006.txt   4  51   7  7       0    1     0          0
  ICE_GB/S1A-007.txt   2  59   8  8       0   10     3          0
  ICE_GB/S1A-008.txt  13  60   3 13       0    3     0          0
  ICE_GB/S1A-009.txt  12  41   4 10       0    3     2          0
  ICE_GB/S1A-010.txt   4  81   0 10       0    1     3          0
[ reached max_ndoc ... 490 more documents, reached max_nfeat ... 41,393 more features ]
```

If text categories are irrelevant, we can obtain the global rank-frequency distribution of all tokens as follows:

```
textstat_frequency(dfmat_raw, n = 20)
```

```
   feature frequency rank docfreq group
1        :    150119    1     500   all
2        >    122667    2     500   all
3        <    122664    3     500   all
4        1     73559    4     500   all
5      the     59586    5     500   all
6        a     55805    6     500   all
7        ,     54219    7     500   all
8       of     29961    8     500   all
9      and     27640    9     500   all
10      to     27627   10     500   all
11       '     21433   11     490   all
12       .     20504   12     201   all
13      in     19404   13     500   all
14       i     19149   14     418   all
15       b     18030   15     273   all
16    that     17204   16     500   all
17      it     16810   17     500   all
18     you     13805   18     385   all
19       s     13325   19     490   all
20      is     12833   20     498   all
```

Something is clearly off here: The most frequent 'words' in ICE-GB are apparently `:`, `>`, `<` and `1`. These tokens stem from the corpus markup, in particular from the text unit headers of the form `< ICE-GB:S1A-001 #1 : 1 : A >`, which occur at the beginning of every single text unit. Note also that `dfm()` converts all tokens to lower case by default. The speaker labels *A* and *B* are therefore counted together with the article *a* and the letter *b*: More than half of the 55,805 instances of *a* are, in fact, speaker labels!

Removing markup

Since all markup in ICE is enclosed in angle brackets, we can remove it with a **regular expression**, i.e., a search pattern that describes the sequences of characters we want to find. The pattern `< [^<>]* >` below can be read as 'an opening bracket `<`, followed by any number of characters that are not brackets (`[^<>]*`), followed by a closing bracket `>`'. Don't worry if this looks cryptic – regular expressions are the topic of the next unit.

In addition, ICE encodes special characters by means of short codes enclosed by `&` and `;`, such as `‘` for the quotation mark ‘ or `&pound-sign;` for £. We remove these as well.[3]

[3] This also splits up the few words that contain accented letters (e.g., *blasé* for *blasé*), which is negligible for our purposes.

```
# 1. Paste the tokens of each text back together into one long string
ICE_GB_text <- sapply(as.list(ICE_GB), paste, collapse = " ")

# 2. Replace every piece of markup <...> and every special character code
 ↪  &...; with an empty space
ICE_GB_text <- gsub("< [^<>]* >", " ", ICE_GB_text)
ICE_GB_text <- gsub("& [A-Za-z0-9-]+ ;", " ", ICE_GB_text)

# 3. Split the texts into tokens again, dropping punctuation, symbols and
 ↪  numbers
ICE_GB_clean <- tokens(ICE_GB_text,
                       remove_punct = TRUE,
                       remove_symbols = TRUE,
                       remove_numbers = TRUE)
```

> **!** Important
>
> Only use the cleaned version `ICE_GB_clean` for counting words. For concordancing, it is best to stick with the original `ICE_GB` object: The markup provides valuable context (e.g., speakers and pauses), and the token positions in the `from` and `to` columns refer to the original corpus files.

The frequency list now looks much more plausible:

```
dfmat <- dfm(ICE_GB_clean)

textstat_frequency(dfmat, n = 20)
```

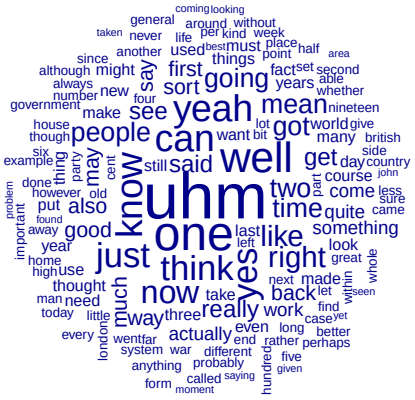
```
   feature frequency rank docfreq group
1      the     59586    1     500   all
2       of     29961    2     500   all
3      and     27640    3     500   all
4       to     27627    4     500   all
5        a     22789    5     500   all
6       in     19404    6     500   all
7        i     19027    7     418   all
8     that     17204    8     500   all
9       it     16809    9     500   all
10     you     13805   10     385   all
11       s     13297   11     490   all
12      is     12833   12     498   all
13     for      8301   13     499   all
14     was      8051   14     479   all
15      on      7394   15     500   all
```

```
16       be       7272   16      500   all
17       uh       7255   17      270   all
18       as       6743   18      500   all
19     this       6317   19      499   all
20     have       6040   20      500   all
```

A popular means of visualisation are wordclouds. As the frequency list shows, the most frequent words are function words such as *the*, *of* and *and*. To obtain a more informative picture, we can remove them using a list of so-called **stopwords** first. We also drop very short tokens with fewer than three characters, most of which are clitics that have been split off during tokenisation (e.g., *s* from *'s* or *ve* from *'ve*):

```
dfmat_content <- dfm_remove(dfmat, pattern = stopwords("en"))
dfmat_content <- dfm_select(dfmat_content, min_nchar = 3)

textplot_wordcloud(dfmat_content, max_words = 150)
```



Zipf's Law

Plotting token frequencies against frequency ranks yields the famous **Zipfian curve** – a small number of high-frequency types followed by a long tail of extremely rare ones:

> Detour: Zipf's Law
>
> "George Kingsley Zipf (Zipf 1949) identified one of the most fundamental statistical regularities in linguistics. It becomes evident when frequency and frequency rank are examined side by side (see the plots below). Here, **frequency rank** refers to a word's position in a frequency hierarchy: Rank 1 denotes the most frequent word, Rank 2 the second most frequent, and so on.
> We denote the frequency of a word by f and its frequency rank by r. Given a free parameter α and a normalising constant C, Zipf proposed the following power-law relationship:

$$f = \frac{C}{r^\alpha}$$

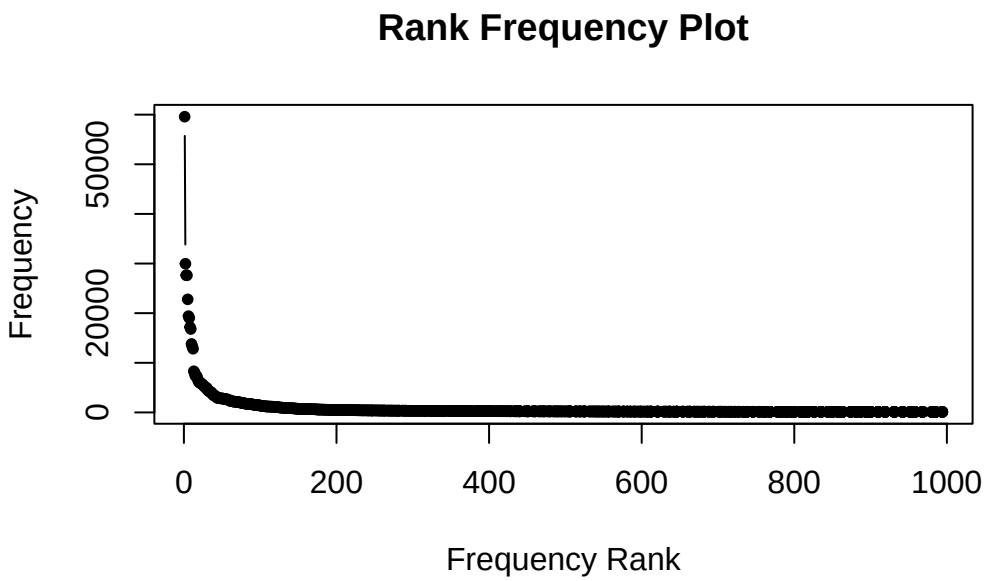
In concrete terms, the frequency of a word is inversely proportional to its frequency rank raised to the power of α. For $\alpha = 1$, the second most frequent word occurs about half as often as the most frequent one, the third most frequent word about a third as often, and so on. In log-space, this relationship becomes linear (additive), with slope $-\alpha$:

$$\begin{aligned}\log(f) &= \log \frac{C}{r^\alpha} \\ &= \log C - \alpha \log r.\end{aligned}$$

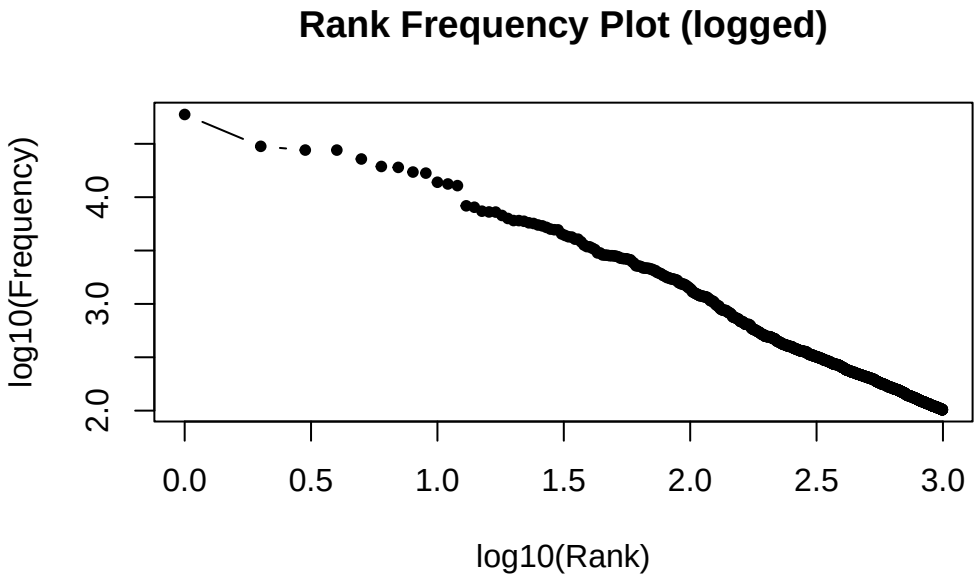
For further details and extensions, see Baayen (2001).

```
# Check and plot distribution of the 1000 most frequent words in ICE-GB
freq <- textstat_frequency(dfmat, n = 1000)

plot(freq$rank, freq$frequency,
     type = "b", pch = 20,
     xlab = "Frequency Rank", ylab = "Frequency",
     main = "Rank Frequency Plot")
```




```
# Log-scale
plot(log10(freq$rank), log10(freq$frequency),
     type = "b", pch = 20,
     xlab = "log10(Rank)", ylab = "log10(Frequency)",
     main = "Rank Frequency Plot (logged)")
```

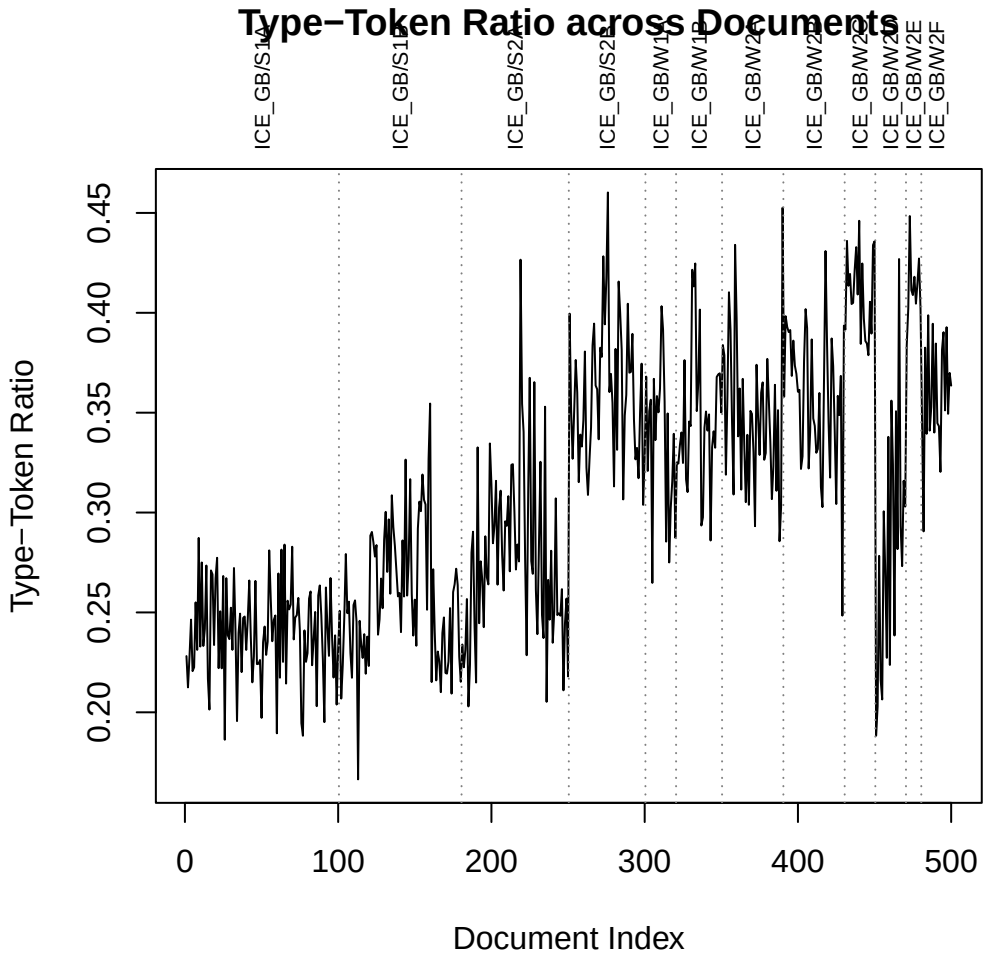


Lexical diversity

Finally, we will inspect the lexical diversity of the ICE texts by computing their type-token ratios (TTR). There is a notable TTR spike from document 251 onwards, which corresponds to category S2B (scripted monologues, such as broadcast news and speeches). The TTR remains high across written text categories up until approx. document 450 (instructional writing, W2D), which is characterised by a significant TTR drop. Subsequently, TTRs rise again in the last two text types, i.e., persuasive writing (W2E) and creative writing (W2F).

```
ttr_stats <- textstat_lexdiv(ICE_GB_clean, measure = "TTR")

plot(ttr_stats$TTR,
     type = "l",
     main = "Type-Token Ratio across Documents",
     xlab = "Document Index",
     ylab = "Type-Token Ratio")
```



Advanced: More on dispersion

The empirical study of dispersion has attracted a lot of attention in recent years (Sönning 2024; Gries 2020). A reason for this is the necessity of finding a dispersion measure that is minimally correlated with token frequency. One such measure is the Kullback-Leibler divergence KLD, which comes from the field of information theory and is closely related to entropy.
Mathematically, KLD measures the difference between two probability distributions p and q.

$$KLD(p \parallel q) = \sum_{x \in X} p(x) \log_2 \frac{p(x)}{q(x)} \tag{1}$$

Let f denote the overall frequency of a keyword in the corpus, v its frequency in each corpus part, s the sizes of each corpus part (as fractions) and n the total number of corpus parts. We thus compare the posterior (= "actual") distribution of keywords $\frac{v_i}{f}$ for $i = 1, ..., n$ with their prior distribution, which assumes all words are spread evenly across corpus parts (hence the division by s_i).

$$KLD = \sum_{i=1}^{n} \frac{v_i}{f} \times \log_2 \left(\frac{v_i}{f} \times \frac{1}{s_i} \right) \tag{2}$$

Corpus parts in which the keyword does not occur ($v_i = 0$) contribute nothing to the sum, since $0 \times \log_2 0$ is defined as 0.
In R, let's calculate the dispersion of the verbs *eat*, *drink*, and *sleep* across the text categories of ICE-GB, using the results from `query3`.

```
# Convert all keywords to lower case, so that 'Drink' and 'drink' are
 ↪  counted together
query3$keyword <- tolower(query3$keyword)
table(query3$keyword)
```

```
drink   eat sleep
   52    53    43
```

```
# Extract text categories from the document names (e.g., "S1A-006.txt"
 ↪  -> "S1A")
query_registers <- separate_wider_delim(query3, cols = docname, delim =
 ↪  "-", names = c("Text_category", "File_number"))

# Get separate data frames for each verb
eat <- filter(query_registers, keyword == "eat")
drink <- filter(query_registers, keyword == "drink")
sleep <- filter(query_registers, keyword == "sleep")

## Get frequency distribution across text categories (v)
v_eat <- table(eat$Text_category)
v_drink <- table(drink$Text_category)
v_sleep <- table(sleep$Text_category)

## Get total frequencies (f)
f_eat <- nrow(eat)
f_drink <- nrow(drink)
f_sleep <- nrow(sleep)
```

Next, we need the sizes of the corpus parts (s). We measure them in words, using the cleaned corpus from above, so that the markup does not distort the counts.

```
# Count the number of words in each file and extract the text
 ↪  categories
corpus_sizes <- tibble(docname = docnames(ICE_GB_clean),
                       n_words = ntoken(ICE_GB_clean)) %>%
  separate_wider_delim(cols = docname, delim = "-", names =
   ↪  c("Text_category", "File_number")) %>%
  group_by(Text_category) %>%
  summarise(n_words = sum(n_words))

# Convert word counts to proportions (s)
s <- corpus_sizes$n_words / sum(corpus_sizes$n_words)
names(s) <- corpus_sizes$Text_category

round(s, 3)
```

```
ICE_GB/S1A ICE_GB/S1B ICE_GB/S2A ICE_GB/S2B ICE_GB/W1A ICE_GB/W1B ICE_GB/W2A
     0.193      0.161      0.145      0.102      0.040      0.057      0.080
ICE_GB/W2B ICE_GB/W2C ICE_GB/W2D ICE_GB/W2E ICE_GB/W2F
     0.082      0.040      0.040      0.020      0.040
```

Since all ICE texts contain approximately 2,000 words each, these proportions are almost identical to the proportions of files per text category. For corpora with texts of unequal length, however, it is essential to measure corpus part sizes in words.
Finally, we plug everything into Equation 2. Since the text categories without any hits contribute nothing to the sum, we only need the values of s for those categories in which the respective verb actually occurs. We can select them by name, e.g., `s[names(v_eat)]`.

```
# Compute KLD for each verb
kld_eat <- sum(v_eat/f_eat * log2(v_eat/f_eat * 1/s[names(v_eat)]));
 ↪  kld_eat
```

```
[1] 0.7058621
```

```
kld_drink <- sum(v_drink/f_drink * log2(v_drink/f_drink *
 ↪  1/s[names(v_drink)])); kld_drink
```

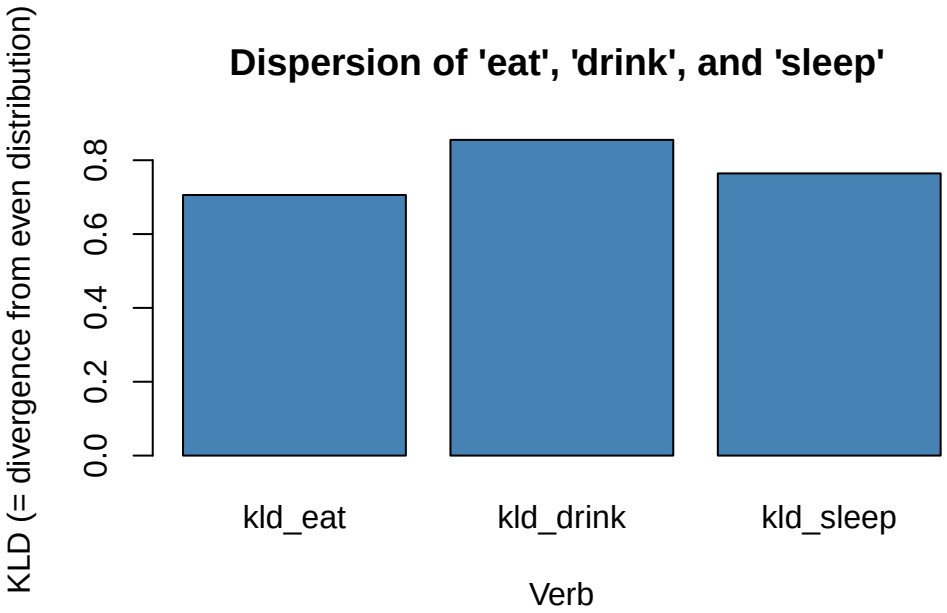
```
[1] 0.8551455
```

```
kld_sleep <- sum(v_sleep/f_sleep * log2(v_sleep/f_sleep *
 ↪  1/s[names(v_sleep)])); kld_sleep
```

```
[1] 0.7643423
```

```r
# Plot
kld_df <- data.frame(kld_eat, kld_drink, kld_sleep)

barplot(as.numeric(kld_df), names.arg = names(kld_df), col =
 ↪  "steelblue",
        xlab = "Verb", ylab = "KLD (= divergence from even
         ↪  distribution)", main = "Dispersion of 'eat', 'drink', and
         ↪  'sleep'")
```



The plot indicates that *drink* is the most unevenly distributed verb out of the three considered (high KLD ~ low dispersion), whereas *eat* appears to be slightly more evenly distributed across text categories. The verb *sleep* assumes an intermediary position.

Alternative concordancing software

There is a wide variety of concordancing software available, both free and paid. Among the most popular options are AntConc (Anthony 2020) and SketchEngine (Kilgarriff et al. 2004). However, as Schweinberger (2024) notes, the exact processes these tools use to generate output are not always fully transparent, making them something of a "black box." In contrast, programming languages like R or Python allow researchers to document each step of their analysis clearly, providing full transparency from start to finish.

References

Anthony, Lawrence. 2020. *AntConc (Version 3.5.9)*. Tokyo, Japan: Waseda University. https://www.laurenceanthony.net/software.

Baayen, R. Harald. 2001. *Word Frequency Distributions*. Vol. 18. Dordrecht: Kluwer.

Benoit, Kenneth, Kohei Watanabe, Haiyan Wang, Paul Nulty, Adam Obeng, Stefan Müller, and Akitaka Matsuo. 2018. "Quanteda: An r Package for the Quantitative Analysis of Textual Data." *Journal of Open Source Software* 3 (30): 774. https://doi.org/10.21105/joss.00774.

Gries, Stefan Thomas. 2020. "Analyzing Dispersion." In *A Practical Handbook of Corpus Linguistics*, edited by Magali Paquot and Stefan Thomas Gries, 99–118. Cham: Springer.

Kilgarriff, Adam, Pavel Rychly, Pavel Smrz, and David Tugwell. 2004. "ITRI-04-08 the Sketch Engine." *Information Technology* 105: 116.

Lange, Claudia, and Sven Leuckert. 2020. *Corpus Linguistics for World Englishes: A Guide for Research*. New York: Taylor; Francis.

McEnery, Tony, Richard Xiao, and Tono Yukio. 2006. *Corpus-Based Language Studies: An Advanced Resource Book*. London: Routledge.

Schweinberger, Martin. 2024. *Concordancing with r*. 2024.05.07 ed. Brisbane: The Language Technology; Data Analysis Laboratory (LADAL). https://ladal.edu.au/kwics.html.

Sönning, Lukas. 2024. "Evaluation of Keyness Metrics: Performance and Reliability," Corpus Linguistics and Linguistic Theory, 20 (2): 263–88. https://doi.org/10.1515/cllt-2022-0116.

Wulff, Stefanie, and Paul Baker. 2020. "Analyzing Concordances." In *A Practical Handbook of Corpus Linguistics*, edited by Magali Paquot and Stefan Th. Gries, 161–79. Cham: Springer International Publishing. https://doi.org/10.1007/978-3-030-46216-1_8.

Zipf, George Kingsley. 1949. *Human Behavior and the Principle of Least Effort*. Oxford, England: Addison-Wesley Press.